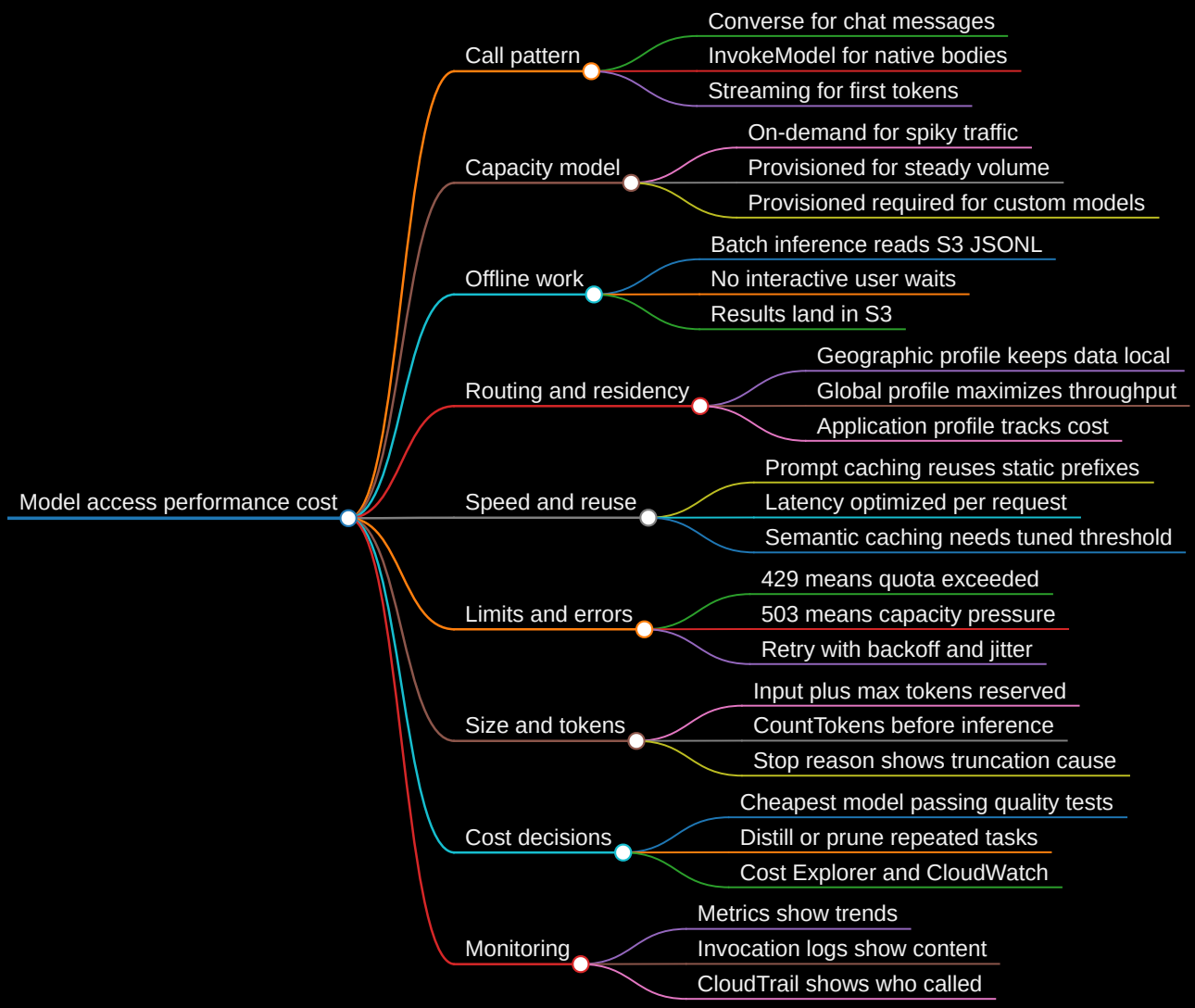
Model access, performance, and cost: how you call the model and how you pay

After this module, you can decide how an application calls a foundation model, how that call is throttled, and how it is paid for. You can read a long scenario, find the one constraint that decides the answer, and reject each wrong option with one reason. Most errors in this area come from mixing four layers: the API, the capacity model, the routing feature, and the cost tool. Keep those layers apart and the answer usually becomes clear.



The deciding constraint

Start at the scenario, not at the service names. Ask four questions in order.

1. Is a user waiting for the answer?
2. Is the volume steady or spiky?
3. Must the data stay in one geography?
4. Is the model customized?

Each answer removes whole branches of options, so you rarely need to compare every choice in detail.

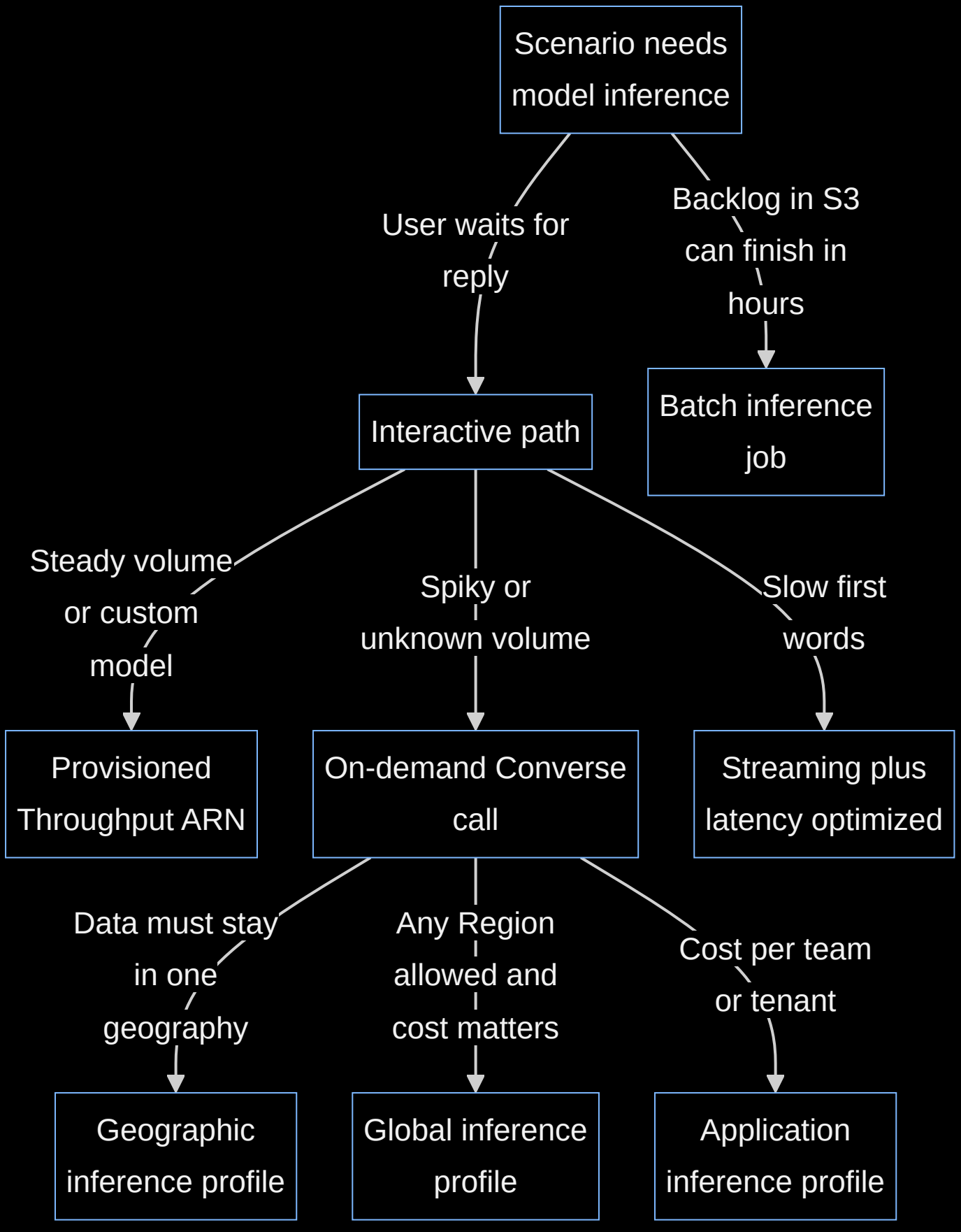


Figure: The first split is whether a person waits for the answer.

Use these signal words to move down the tree.

- Rule: Words such as chat, live agent, or typing point to the interactive path.
- Rule: Words such as overnight, backlog, S3 files, or by morning point to batch inference.

- Rule: Words such as guaranteed, reserved, or steady high volume point to Provisioned Throughput.
- Rule: The words customized model or fine-tuned model mean you must purchase Provisioned Throughput.
- Rule: Words such as spike, sale, or unpredictable point to on-demand plus quota planning.
- Rule: Words such as residency, jurisdiction, or stay in the country point to a geographic profile.
- Rule: Words such as maximize throughput with no residency limit point to a global profile.
- Rule: Words such as chargeback, cost per team, or tags point to an application inference profile.

Exam tip: The word steady in a stem usually supports reserved capacity, while the word nightly usually supports batch. Read the time window and the user type before you read the options.

Converse API

What it is. The Converse API sends a conversation to a model through one request shape. It runs on the bedrock-runtime endpoint. The request carries messages, an inference configuration with maxTokens, temperature, topP, and stop sequences, and optional tool and guardrail settings. The response carries output, stopReason, usage, and metrics.

Why it matters. The same code path can call several models that support messages. Model-specific settings go in a separate field, so you change less code when you switch models.

Rule: Choose Converse for chat and multi-turn messages, especially when you may change models later. Choose it when you want one response shape for every caller.

Trap: Converse does not remove every model difference. Provider-specific parameters still belong in the additional model request field, and not every

model supports messages.

Pairs with. Guardrails in the request, tool use, ConverseStream for live output, and the usage fields for cost tracking.

Limits: The usage object reports inputTokens, outputTokens, and totalTokens. When prompt caching is on, total input equals inputTokens plus cacheReadInputTokens plus cacheWriteInputTokens. Context window size per model: Not in source.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference.html , https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html

InvokeModel and InvokeModelWithResponseStream

What it is. These base operations send a model-native JSON body. You build the body for each provider and model family. The response is also model-specific. The streaming variant returns the output as an event stream.

Rule: Choose InvokeModel when you need a parameter that only the native body exposes, or when the model does not use the message format. Choose the streaming variant when the native body is required and users read output live.

Trap: Do not pick InvokeModel only because it sounds lower level. Each model switch then needs a new body format. Also, calling the streaming variant and reading only the final body removes the benefit of streaming.

Pairs with. The model ID or an inference profile in modelId, a provisioned model ARN in modelId, and Bedrock invocation logging for supported calls.

Limits: Body schema per model: Not in source for this module. Check the model parameter reference before you write the payload.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference.html

Streaming with ConverseStream

What it is. Streaming sends output in pieces while the model generates it. The TimeToFirstToken metric applies to streaming calls. Streaming changes when a

user sees the first words. It does not reduce the work the model does.

Rule: Choose streaming when a person reads the answer as it forms. Choose a non-streaming call when a program needs the complete answer before it acts.

Trap: Streaming does not fix throttling, and it does not make a slow model fast. A proxy or gateway that buffers the response also hides the stream from the browser.

Pairs with. Latency optimized inference, the TimeToFirstToken metric, and an API front end that can pass chunks through.

Limits: Gateway streaming rules for API Gateway: Not in source for this module. Check the API Gateway documentation before you choose the API type.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference.html , Sundog slides page 286 (TimeToFirstToken in streaming)

On-demand inference

What it is. On-demand inference charges per token and shares model capacity with other customers. Your ceiling is the account quota for each model and Region. Quotas limit tokens per minute, and some models also limit requests per minute.

Rule: Choose on-demand for unknown, variable, or low volume traffic. Choose it when a short wait or a retry during a spike is acceptable.

Trap: On-demand does not guarantee capacity during a spike. Retries and higher client concurrency do not create capacity, and they can add load during the spike.

Pairs with. Token-aware admission control, retries with backoff, cross-Region inference profiles, and Service Quotas increase requests.

Limits: Default quota values: Not in source for this module. Read your values in Service Quotas, because account allocations can differ from published defaults.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/quotas.html , https://docs.aws.amazon.com/bedrock/latest/userguide/scaling-throughput-best-

practices.html

Provisioned Throughput

What it is. You buy Model Units for one model. Each Model Unit sets how many input tokens and output tokens per minute the model can process for you. You choose no commitment, a 1-month commitment, or a 6-month commitment, and longer commitments cost less per hour. Billing continues until you delete the throughput.

Custom models. A customized model must use Provisioned Throughput. You then pass the provisioned model ARN as the modelId in the runtime call.

Rule: Choose Provisioned Throughput for steady high volume with a latency target, or for any customized model. Choose it when the workload runs for most hours of the day.

Trap: Do not buy it for a weekly or nightly job, because you pay every hour until you delete it. Do not pair it with cross-Region inference profiles, because profiles do not support it. Batch inference also does not run on provisioned models.

Pairs with. CloudWatch utilization dashboards, Cost Explorer for attributing model cost, and the runtime quota planning that starts from tokens per minute.

Limits: Tokens per minute for each Model Unit: Not in source. Price per hour and commitment discounts: Not in source. Ask the AWS account team for these numbers.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html , https://docs.aws.amazon.com/bedrock/latest/userguide/cross-region-inference.html , Sundog slides page 291

Batch inference

What it is. Batch inference reads many prompts from files in Amazon S3 and writes responses back to S3. The job runs asynchronously. You can format the input for either the InvokeModel or the Converse request style.

Limits in the docs. Batch inference is not supported for provisioned models. Jobs that use the Converse invocation type do not accept tool configuration, and such records fail with a 400 error. Structured output works with both invocation types.

Rule: Choose batch when no user waits and records sit in S3. A completion window of hours must be acceptable. Use it for backfills, nightly summaries, and bulk classification.

Trap: Do not choose batch for live chat. Do not expect prompt caching to apply, because the docs say caching is not supported with batch. Do not choose the Converse invocation type when the job needs tools.

Pairs with. S3 input and output locations, a service role that grants access to the buckets, and EventBridge notifications for job state changes.

Limits: Records per job, file size, and job duration: Not in source for this module. Discount versus on-demand: Not in source. The Sundog slides say batch is for submitting many prompts in S3 and getting responses in S3.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/batch-inference.html , https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html , Sundog slides page 290

Cross-Region inference profiles

What it is. A cross-Region inference profile defines a foundation model and the Regions where Bedrock may route your request. A geographic profile keeps routing inside one geography, for example the US, the EU, or APAC. A global profile routes to any supported commercial Region. Traffic stays on the AWS network and is encrypted in transit.

Cost and logging. Geographic profiles use standard pricing. Global profiles are listed at about 10 percent savings in the docs table. There is no extra routing fee, and the price follows the Region you call from. CloudTrail records the inference Region in the additionalEventData field.

Rule: Choose a geographic profile when data residency keeps processing in one geography. Choose a global profile when residency allows any commercial

Region and you want the lower price and the widest capacity.

Trap: A global profile is not a residency tool. A Service Control Policy can block profile Regions. Allow every Region in the profile, or allow the unspecified Region condition for global profiles.

Pairs with. Application inference profiles, Service Control Policies, CloudTrail, and the modelId field in Converse or InvokeModel.

Limits: Profiles do not support Provisioned Throughput. The docs give the US geographic profile as an example that routes across us-east-1, us-east-2, and us-west-2. The complete Region list for each profile: Not in source for this module.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/cross-region-inference.html , Sundog slides pages 304 and 305

Application inference profiles

What it is. An application inference profile tracks usage and cost for the requests that use it. You create it from one foundation model, which tracks one Region, or from a cross-Region system-defined profile, which tracks the Regions in that profile. It accepts tags and returns an ARN that you use as the modelId.

Rule: Choose an application profile when you need cost attribution per application, team, or tenant. Create it from the cross-Region profile when you also need multi-Region routing.

Trap: An application profile is not a capacity purchase. A profile made from a single model keeps single-Region routing, so it does not add failover. Do not expect it to replace Provisioned Throughput.

Pairs with. Cost allocation tags, Cost Explorer reports, and the system-defined profile it copies.

Limits: Quota per account for profiles: Not in source. Required fields are a name and a model source. Description, tags, and client token are optional.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/inference-profiles-create.html

Prompt caching

What it is. Prompt caching reuses a repeated prompt prefix so the model does not process it again. Implicit caching is automatic and best effort. Explicit caching places cache checkpoints in the request. Each checkpoint needs a minimum prefix length. The docs table lists 512 to 4,096 tokens, depending on the model.

Checkpoints are processed in the order tools, then system, then messages. Up to four checkpoints per request apply to the listed Claude models. The time to live is 5 minutes for most listed models. Some models also allow 1 hour, set with a ttl value.

Billing. Cache reads are billed at a lower rate. Cache writes can cost more than standard input tokens. A cache hit is not guaranteed, so read cacheReadInputTokens and cacheWriteInputTokens on every response.

Rule: Choose prompt caching when a long static prefix repeats, such as policy text or a tool schema. Put static content first and the changing user input last.

Trap: Do not put per-user data in a prefix that you expect to repeat. Do not expect a benefit when every prompt is unique. Prompt caching is not a response cache, so it does not return a saved answer.

Pairs with. The cachePoint field in Converse, the cache control field in InvokeModel for Claude models, CloudWatch cache token metrics, and semantic caching for answers.

Limits: Cache hit rate for your workload: Not in source. Batch inference is not supported. Cross-Region inference can be used with caching, and it can increase cache writes at peak demand.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html , Sundog slides pages 293 and 294

Intelligent prompt routing

What it is. Intelligent prompt routing gives one serverless endpoint that sends each request to one of two models from one model family. Bedrock predicts the

response quality of each model for the prompt. A fallback model acts as the baseline, and a response quality difference setting decides when to leave the baseline.

Limits in the docs. The feature is optimized for English prompts. It cannot adjust routing from your application performance data. Default routers use selected Amazon Nova and Meta Llama models, and a configured router must contain exactly two models from the same family.

Rule: Choose intelligent routing when one family holds a cheaper model and a stronger model, the traffic is English, and you want no custom router. Test the quality difference before you set the criterion.

Trap: Do not use it across different model families, because the feature routes inside one family. Do not use it to meet a residency rule or a latency target, because it does not solve either.

Pairs with. Model evaluation for quality tests, Cost Explorer for cost checks, and a custom router in application code when families differ.

Limits: The supported model and Region table is in the docs and changes over time. Response quality difference values in the docs example: 10 percent for a fallback comparison.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-routing.html , Sundog slides pages 288 and 289

Latency optimized inference

What it is. You set performanceConfig with latency set to optimized on a runtime call. The default is standard. The feature is in preview. It keeps the same model, and it needs no fine-tuning or new endpoint.

Fallback and visibility. When the optimized quota for a model is used up, Bedrock serves the request at standard latency and bills it at standard rates. The served setting appears in the API response and in CloudTrail. CloudWatch shows latency-optimized requests under a separate model ID suffix.

Rule: Choose latency optimized when the same supported model must stay, the call is interactive, and you can set the flag on each request. Use it for the live path and leave background calls on standard.

Trap: Do not treat it as extra capacity. It targets response speed, not throughput. Do not assume every model or Region supports it, because the docs list specific models and cross-Region profiles.

Pairs with. Streaming for early words, prompt caching for long prefixes, and cross-Region profiles where the table lists them.

Limits: The docs list a request size limit for one Llama model in optimized mode, and larger requests fall back to standard. The current model list: Not in source for this module.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/latency-optimized-inference.html , Sundog slides page 295

Service quotas and token burndown

What it is. Quotas are set per model and per Region. Token quotas count input tokens, output tokens, and cache write tokens, while cache read tokens do not count. The system deducts input tokens plus max_tokens at the start of a request, and it refunds unused reservation after the response.

Output weighting. Output tokens count at 1 to 1 for most models. The burndown page lists higher output rates for some Anthropic model versions, such as 5 times, 10 times, or 15 times. Check the model card for the rate that applies to your model.

Rule: When throttling appears while request count is under the limit, check token quotas and the max_tokens value first. Lower the reserved amount before you ask for more quota.

Trap: A quota is an upper bound, not a promise that every request runs at once. Quota increases are not automatic, and the requests per minute limit is not the only control for token-heavy traffic.

Pairs with. CountTokens for input estimates, maxTokens for output reservation, retries, and Provisioned Throughput when the steady load is known.

Limits: Default values per model: Not in source. The bedrock-runtime and bedrock-mantle endpoints have separate quota allocations. Daily token quotas also exist, and their values are Not in source.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/quotas-token-burndown.html , https://docs.aws.amazon.com/bedrock/latest/userguide/quotas.html , https://docs.aws.amazon.com/bedrock/latest/userguide/scaling-throughput-best-practices.html

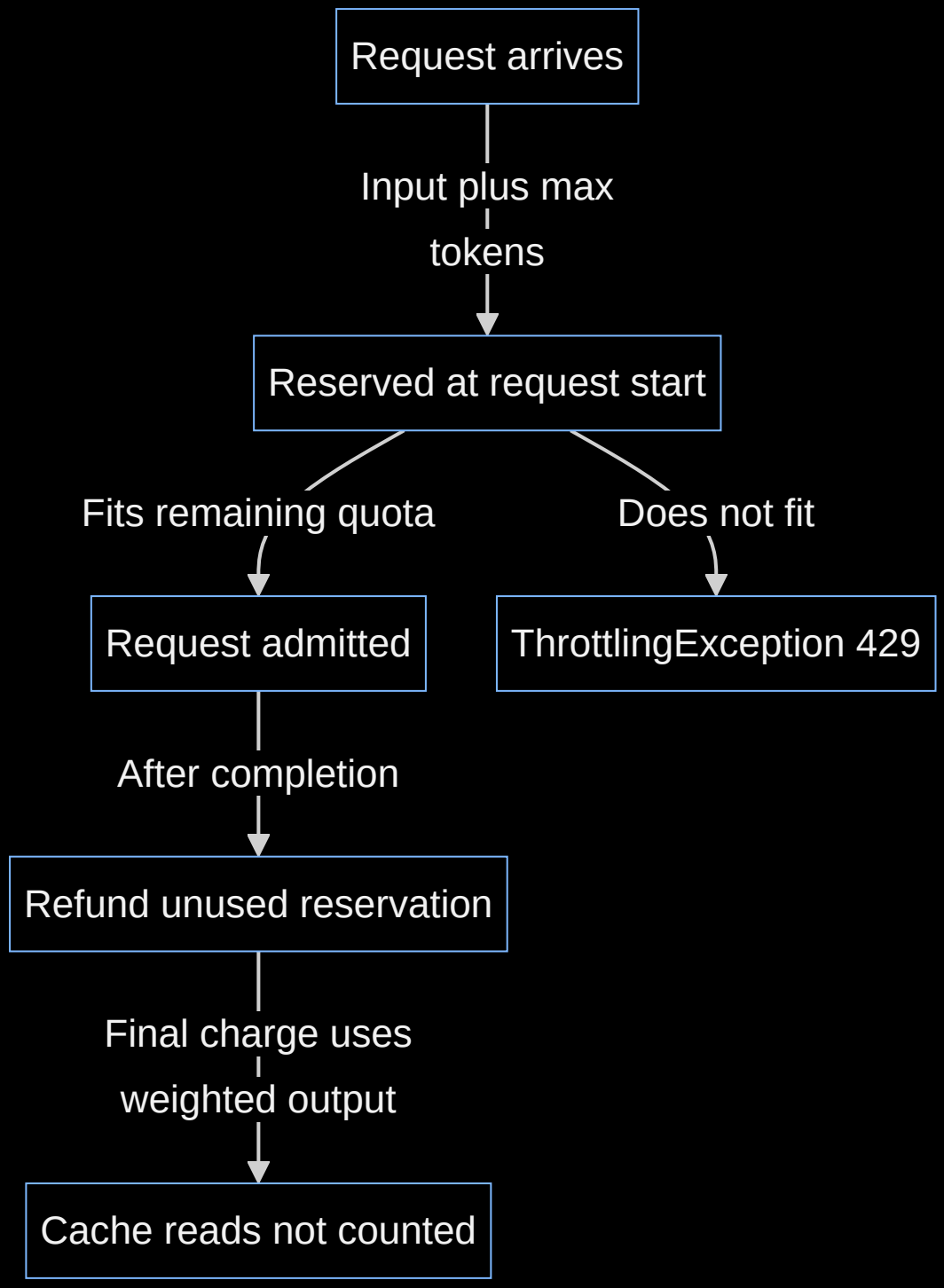


Figure: A large max_tokens value reserves quota before any output exists.

Error codes, throttling, and retry with backoff

What it is. A 429 ThrottlingException means the request exceeded an account quota. A 503 ServiceUnavailable means temporary demand or capacity pressure, and it is not an account quota. A 529 overloaded_error is a transient capacity error. A 400 ValidationError means the input broke a rule, and a 403 AccessDeniedException means the caller lacks permission.

Retry pattern. For 429, 503, and 529, retry with exponential backoff and random jitter. Honor a Retry-After header when one is present. Keep the total attempt count bounded, and count the first attempt in that bound when you use the SDK setting total_max_attempts.

Rule: For a 429, reduce token demand or request a quota increase, then retry with bounded backoff. For a 503 or 529, back off, reduce concurrency, and consider another Region when the model and residency rules allow it.

Trap: Do not retry a ValidationError, because the same input fails again. Do not treat a 503 as a quota problem. Retry storms and extra Lambda concurrency send more load into the same capacity.

Connection trap. NAT Gateways, interface endpoints, and Network Load Balancers drop connections that sit idle for 350 seconds. The client may see a timeout or a long first call after idle time. The docs require both SDK TCP keep-alive and a lower OS keep-alive interval below 350 seconds.

Limits: The SDK default attempt count: Not in source. The six total attempts in the docs example are a sample budget, not a default.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/troubleshooting-api-error-codes.html , https://docs.aws.amazon.com/bedrock/latest/userguide/scaling-throughput-best-practices.html

Max tokens and context windows

What it is. maxTokens in inferenceConfig caps the generated output. The stopReason field explains why generation ended. The docs show max_tokens when the cap was reached, and end_turn when the model finished on its own.

Rule: Set maxTokens to the real answer budget for each task class. If stopReason is max_tokens, raise or change the cap for that class. If the prompt itself is too large, prune input instead.

Trap: Raising maxTokens does not fix an input that is too large for the context window. Setting it to the model maximum wastes quota reservation on every request, and reducing it does not shrink retrieved context.

Pairs with. CountTokens for input size, stop reason logging for diagnosis, and token budgets in the gateway or in application code.

Limits: Context window size for each model: Not in source. The name of the stop reason for input overflow: Not in source for this module. Check the Converse stop reason list before you write a handler for that case.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference.html , https://docs.aws.amazon.com/bedrock/latest/userguide/quotas-token-burndown.html

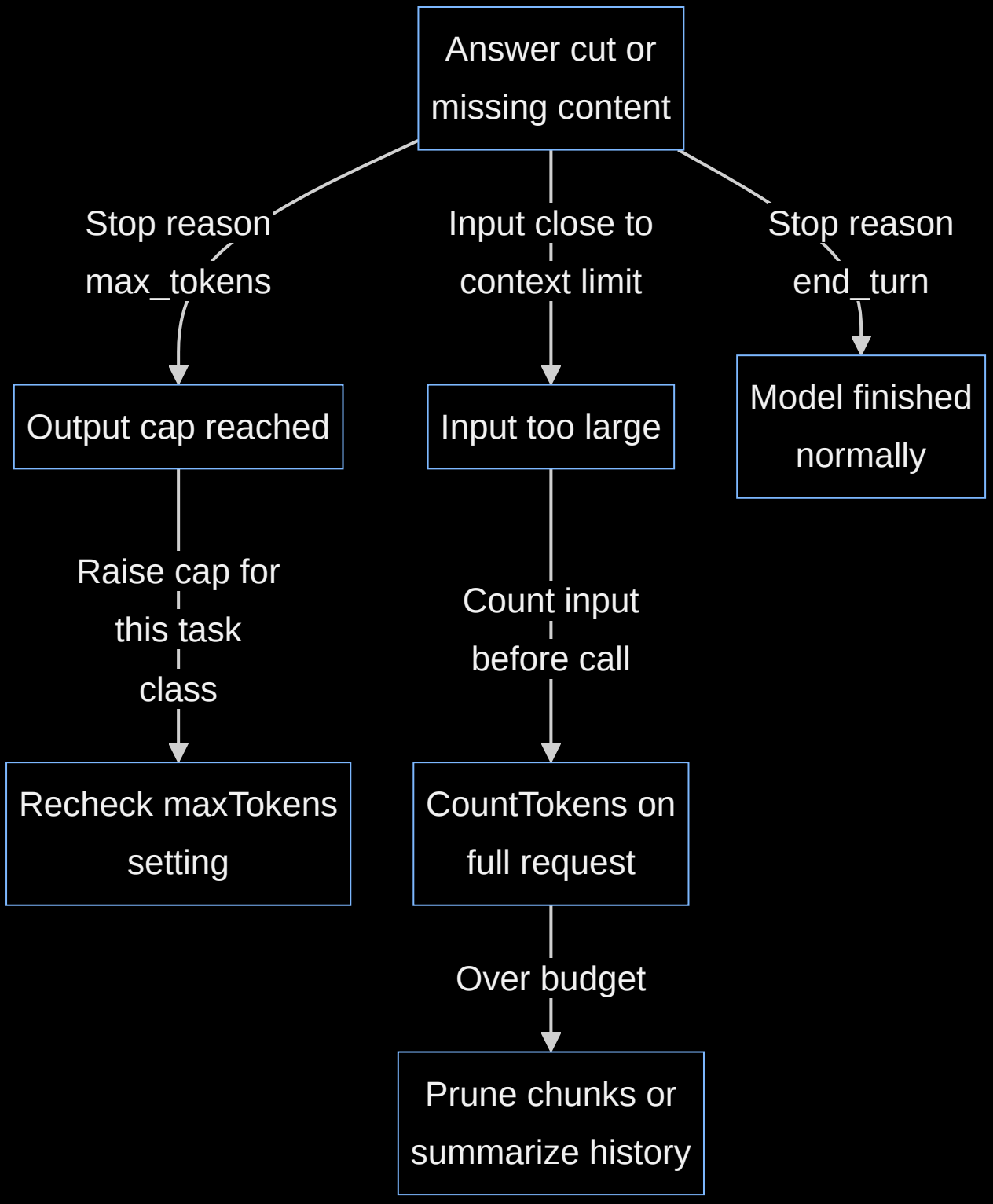


Figure: Read the stop reason before you change any limit.

CountTokens

What it is. CountTokens returns the input token count for a request, for one named model, without running inference. Different models count the same text differently. Sundog course slides state that the call costs nothing.

Rule: Call CountTokens on the full assembled request, for the same model you will invoke. Use the count to reject, prune, or summarize the request before the model runs.

Trap: CountTokens counts input only, so it does not predict output. Do not use the count from one model for another model. Do not treat the count as a replacement for maxTokens.

Pairs with. Prompt pruning, maxTokens, token budgets, and the usage fields that report actual counts after the call.

Limits: Request size limits for CountTokens: Not in source for this module.

Source: Sundog slides pages 285 and 289, https://docs.aws.amazon.com/bedrock/latest/userguide/quotas.html

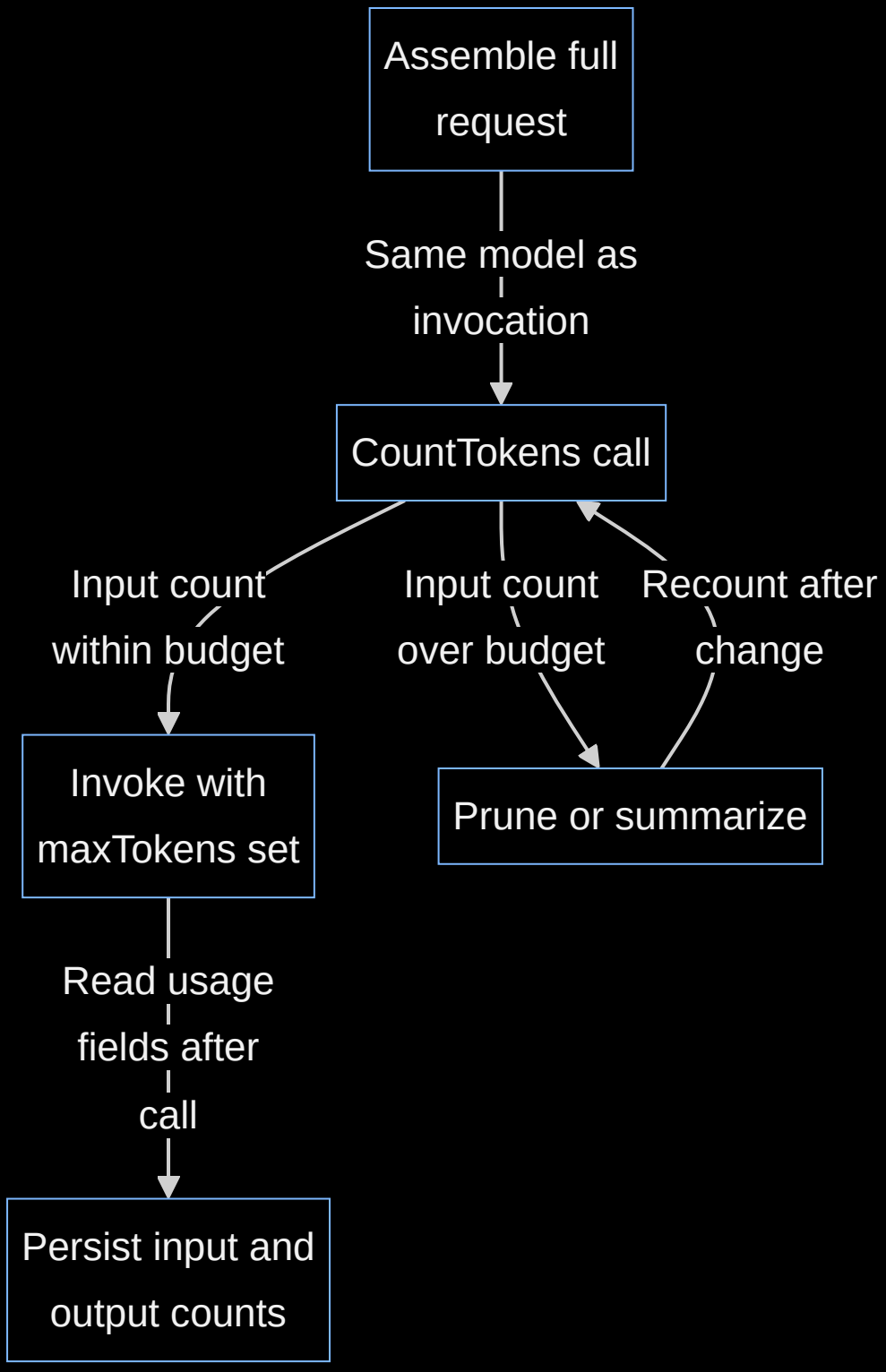


Figure: Count, decide, then invoke. The count covers input only.

Model selection by cost and quality

What it is. Model selection matches a model to each task, then measures quality and cost for that task. Bedrock evaluations can compare models with your own prompts, using human judges, LLM judges, or predefined metrics. Price per token alone does not show cost per correct result.

Rule: Choose the smallest model that passes your acceptance tests for each task. Test each stage of a workflow on its own, because the stages often need

different capability.

Trap: Do not choose by leaderboard score, by list price alone, or by the largest model for every stage. A cheaper model that needs retries can cost more per correct answer.

Pairs with. Intelligent routing inside one family, distillation for narrow repeated tasks, and CloudWatch token metrics for the measured cost.

Limits: Model prices by Region: Not in source for this module. Use the pricing page and your own test results.

Source: Sundog slides pages 288 and 289, https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-routing.html

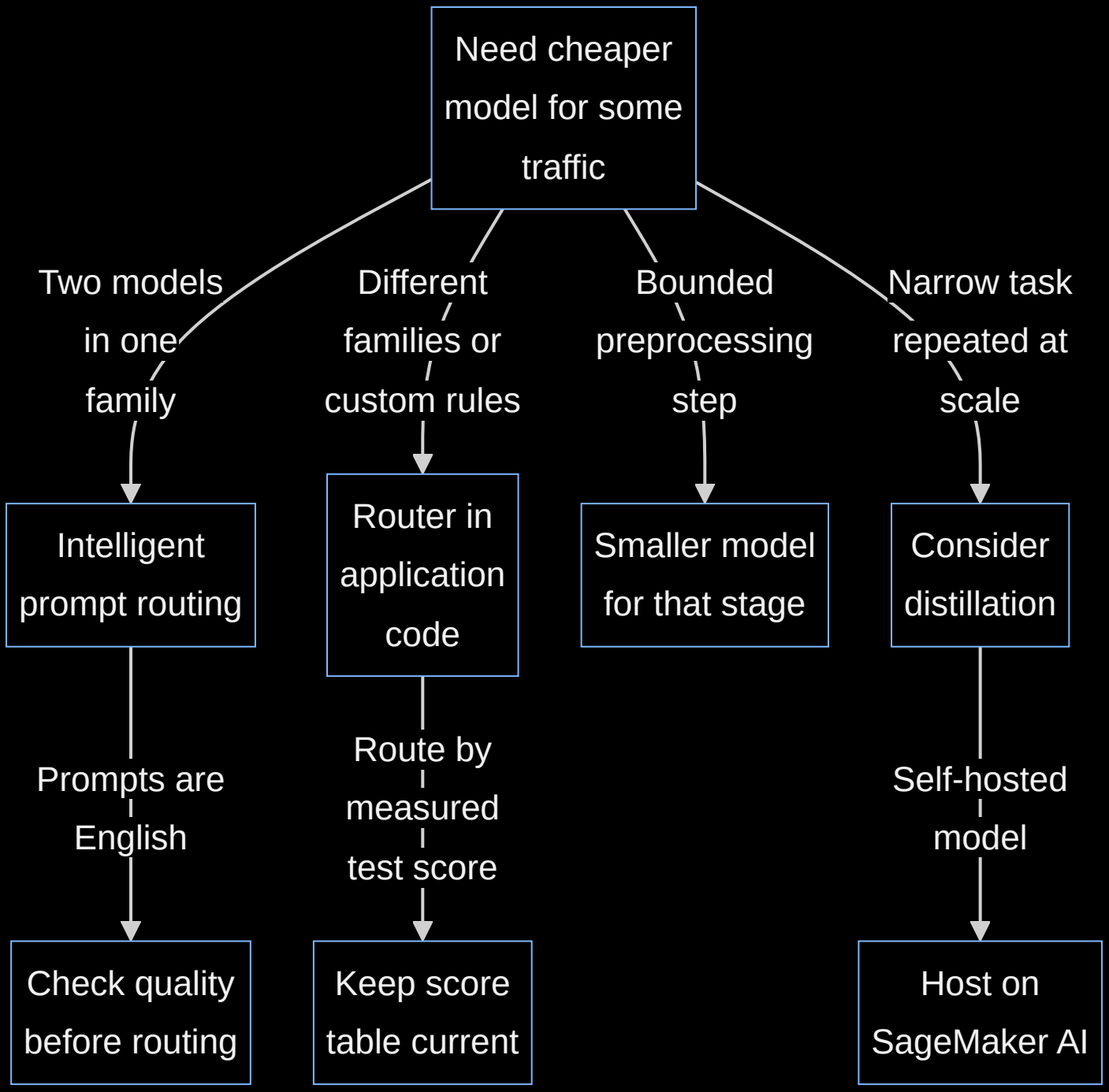


Figure: Pick the mechanism by the family rule first, then by the cost goal.

Distillation, pruning, and smaller models

What it is. Knowledge distillation trains a smaller model from a larger model. Quantization lowers the precision of model weights, and pruning removes weights. The Sundog slides place these methods in the self-hosting part of the course, where teams run model servers such as those in SageMaker AI.

Rule: Use a smaller model for bounded preprocessing, such as classification, chunking, or compression. Consider distillation when you host the model yourself and the task is narrow and repeated at scale.

Trap: Distillation is training work, not a setting you toggle on a Bedrock call. Do not distill before you measure cost and quality. Do not move final generation to a small model when the output quality is the goal.

Pairs with. SageMaker AI hosting, model selection tests, and token pruning in the prompt.

Limits: Bedrock custom distillation product details: Not in source for this module. Size reduction percentages: Not in source.

Source: Sundog slides pages 288 and 319

Semantic caching

What it is. Semantic caching stores embeddings of earlier prompts and their responses. For a new prompt, the application embeds it, searches for nearest neighbors, and returns the cached response when the similarity score passes a threshold. The Sundog slides name ElastiCache for Valkey as a vector store for this pattern.

Rule: Choose semantic caching when many users ask similar questions and the same answer stays correct for all of them. Tune the threshold, and set an expiry time for the cached entries.

Trap: Do not use it for personal, account-specific, or fast-changing answers. Do not confuse it with prompt caching. Prompt caching reuses prefix tokens inside the model. Semantic caching returns a saved answer outside the model.

Pairs with. ElastiCache for Valkey or MemoryDB, and edge caching for deterministic request fingerprints.

Limits: Similarity threshold value: Not in source. The overhead of embedding and search must be lower than the saved inference cost, which you measure in your own tests.

Source: Sundog slides pages 292, 294, and 571

Cost monitoring with CloudWatch and Cost Explorer

What it is. The AWS/Bedrock namespace in CloudWatch publishes runtime metrics. Useful ones include InputTokenCount, OutputTokenCount, CacheReadInputTokenCount, CacheWriteInputTokenCount, InvocationLatency, TimeToFirstToken for streaming, and InvocationThrottles. Cost Explorer shows spend by service, and tags let you attribute model cost to business functions.

Rule: Use CloudWatch for token, latency, and throttle trends. Use Cost Explorer for money by service or tag. Use invocation logs to inspect content, and use CloudTrail to see who called the API and when.

Trap: Metrics do not show prompt or response text. Cost Explorer does not show throttles. Do not read TimeToFirstToken for a non-streaming call, because the metric applies to streaming calls.

Pairs with. Application inference profiles for tags, Cost Anomaly Detection for unusual spend, and dashboards that compare token growth with request growth.

Limits: Log destinations, retention, and cost of invocation logs: Not in source for this module. Output tokens per second is a value you calculate from metrics. The exact formula: Not in source here.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/quotas-token-burndown.html , https://docs.aws.amazon.com/bedrock/latest/userguide/inference-profiles-create.html , Sundog slides pages 286, 291, 446, 608, and 609

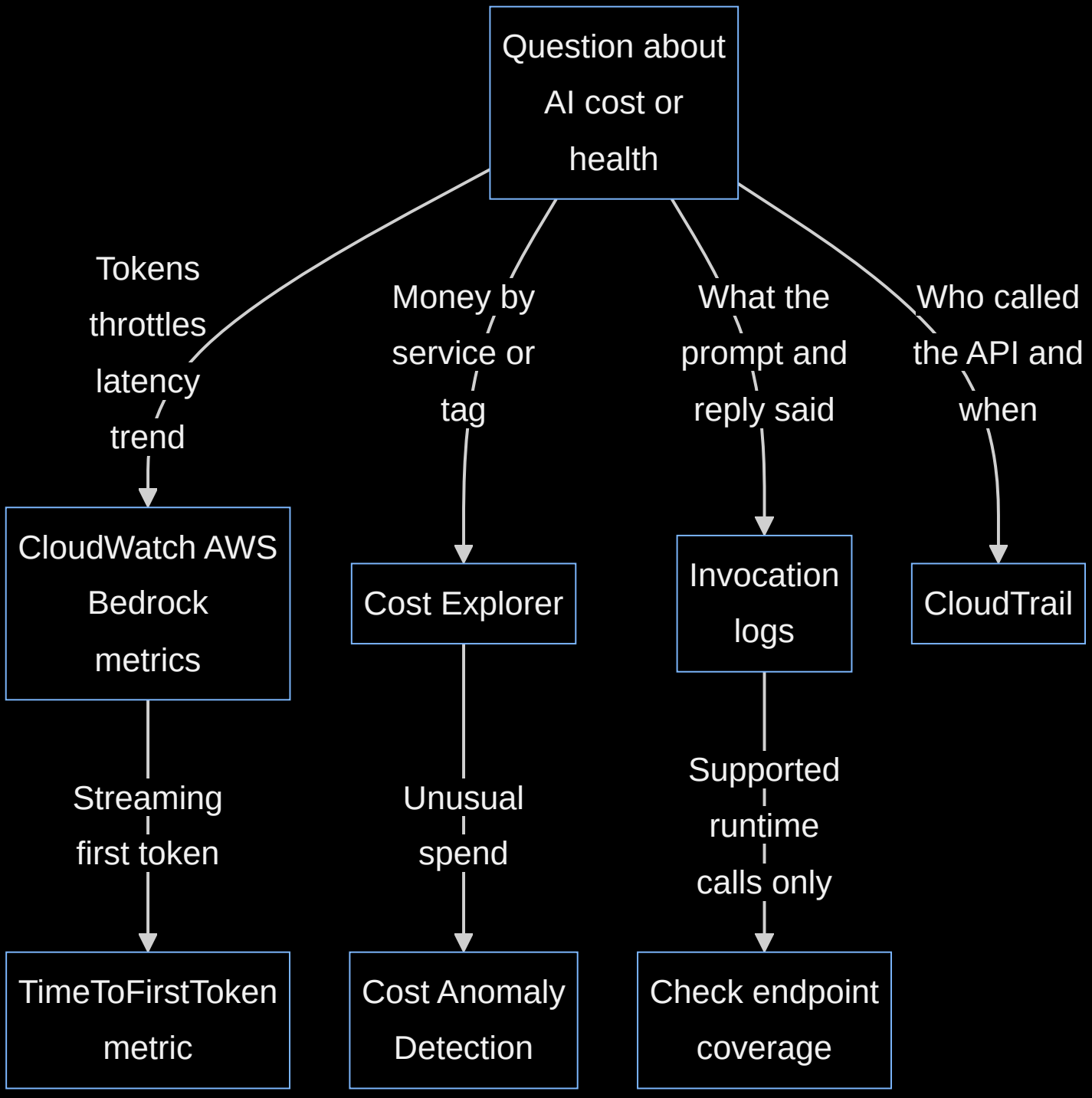


Figure: Each tool answers a different question, so match the tool to the question.

Versus pairs: the confusable choices

Each pair below has one decision tree, a three-row comparison, one question to ask yourself, and one trap. Read the question first. The question usually names the constraint that decides the pair.

Converse versus InvokeModel

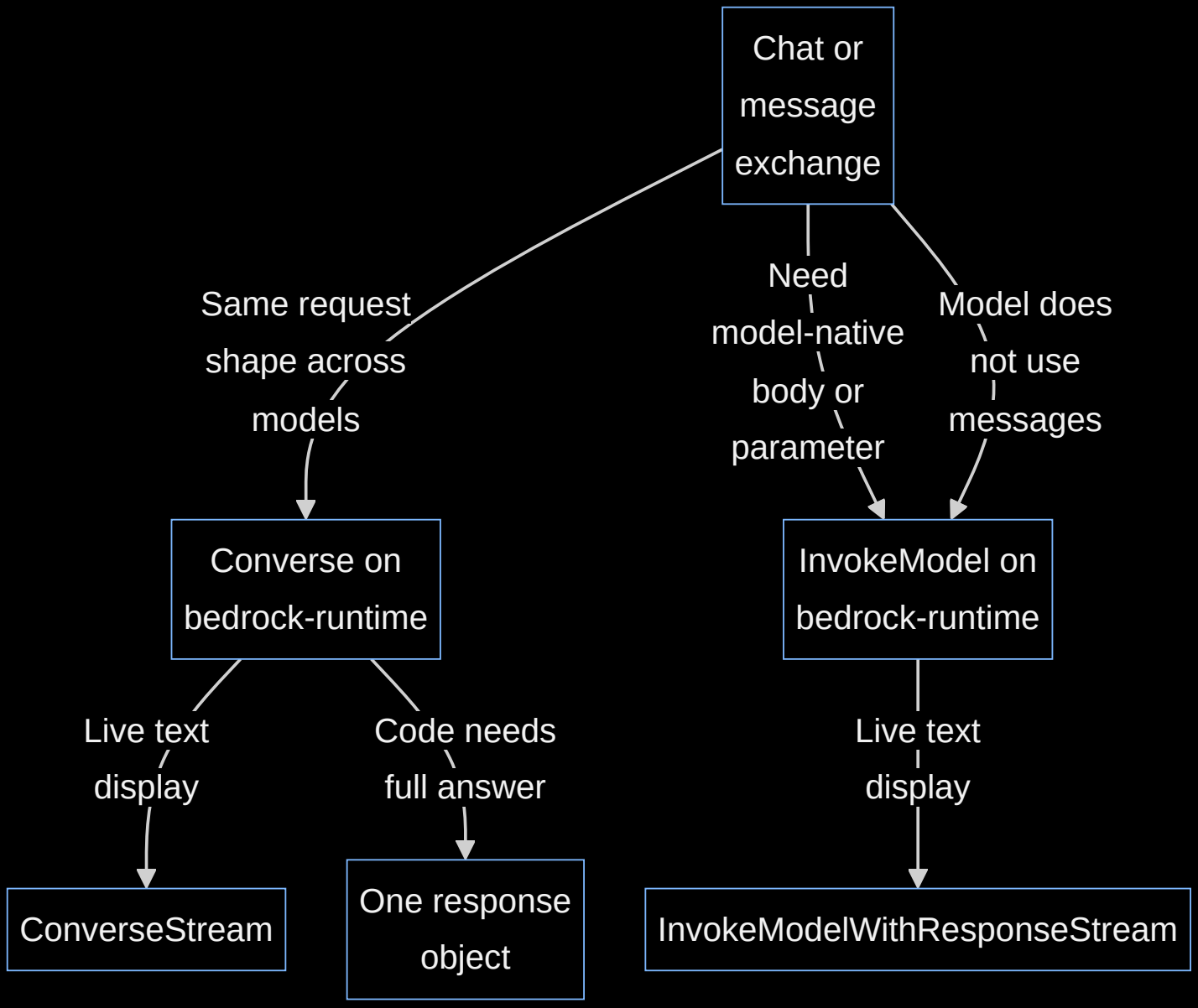


Figure: Pick the request style first, then decide on streaming.

- Purpose: Converse gives one message format for many models. InvokeModel gives direct access to each model body.
- Input: Converse takes messages and an inference configuration. InvokeModel takes a model-specific JSON body.
- Output or effect: Converse returns a normalized envelope with usage and stop reason. InvokeModel returns the body that the model family defines.

The one question: Will the same code path call more than one model family?

Trap: Choosing InvokeModel for a chat app because it looks more direct. The result is one payload parser per model, and a model switch becomes a code change.

On-demand versus Provisioned Throughput

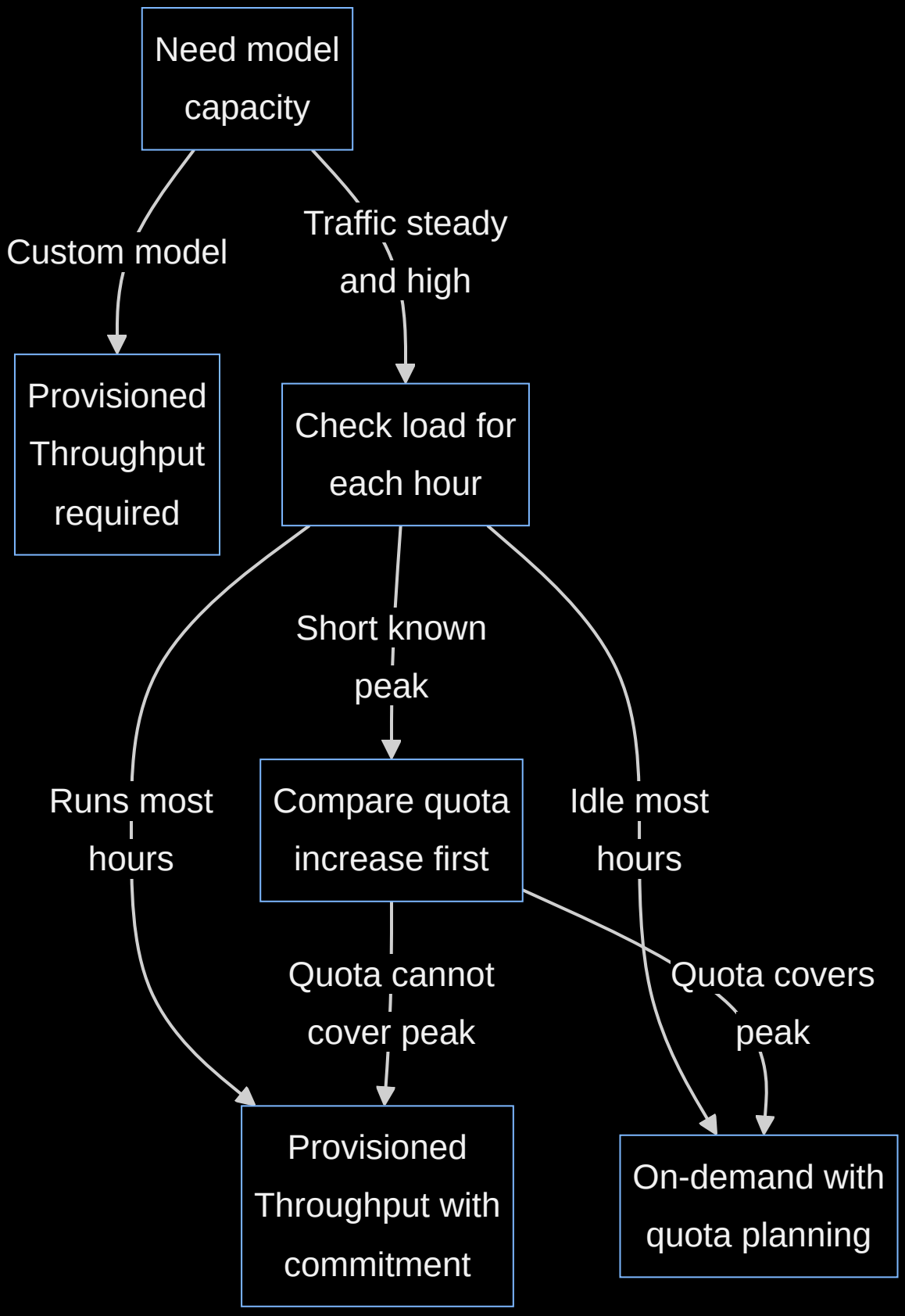


Figure: Hours of use decide whether reserved capacity pays off.

- Purpose: On-demand serves variable load with shared capacity. Provisioned Throughput reserves model units for one model.
- Input: On-demand takes tokens as they come. Provisioned Throughput takes a purchased model unit count and a provisioned model ARN.

- Output or effect: On-demand bills per token and can throttle. Provisioned Throughput bills per hour until deleted and gives a reserved token rate.

The one question: Do you pay for capacity during hours when nobody sends traffic?

Trap: A stem that says guaranteed capacity still needs a check for custom models and for residency. Profiles do not support Provisioned Throughput, so a residency rule can rule it out for cross-Region routing.

Batch inference versus real-time calls

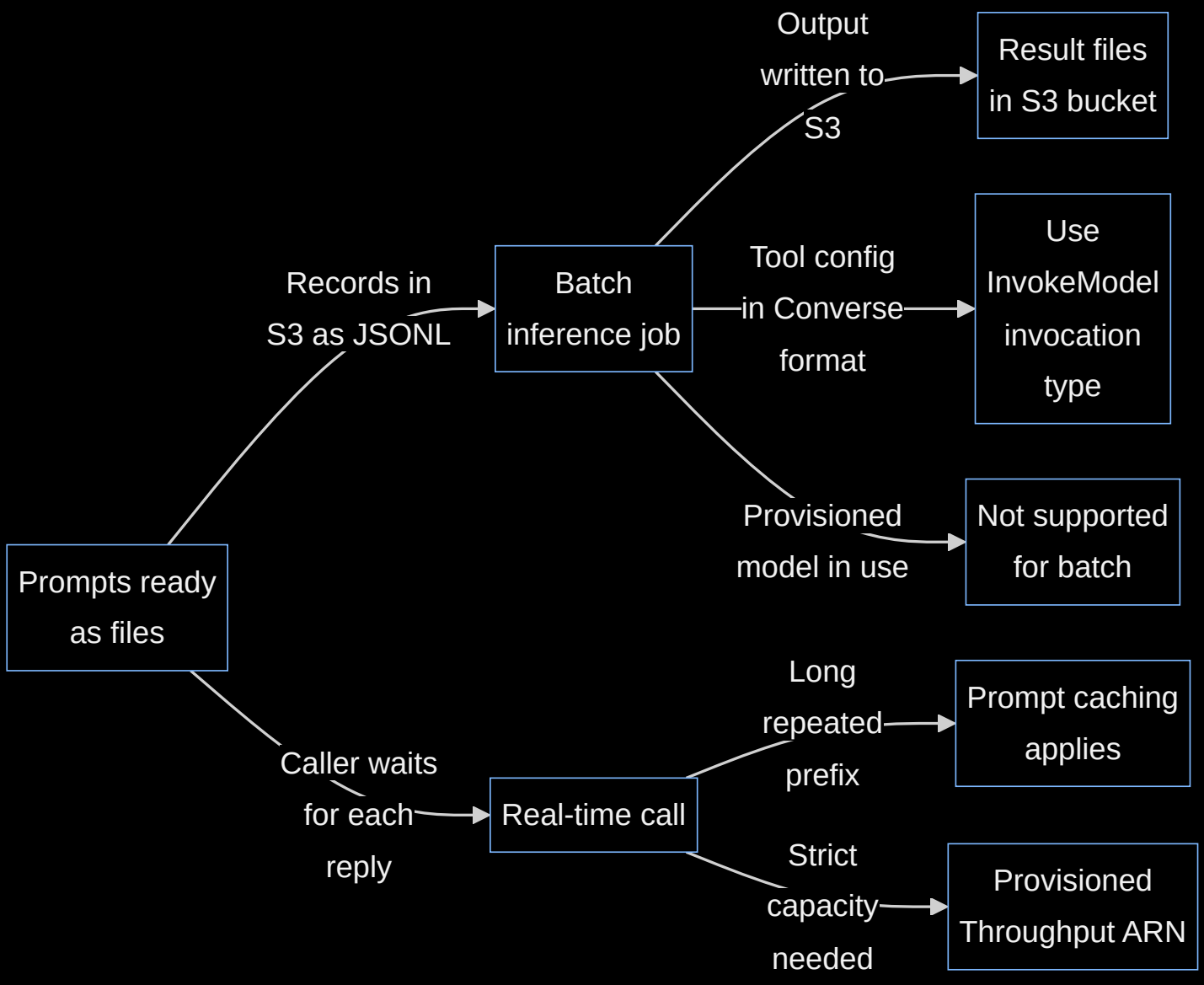


Figure: A waiting user rules out batch, and a file backlog rules out real-time loops.

- Purpose: Batch processes a large backlog asynchronously. Real-time calls answer one request while someone waits.

- Input: Batch reads prompt records from S3. Real-time calls read one request from the application.
- Output or effect: Batch writes result files to S3 after the job. Real-time calls return a response within the request.

The one question: Does a person or a system wait for each answer?

Trap: Using a Lambda loop over S3 files to mimic batch. It adds retry logic, concurrency tuning, and throttling risk that the managed job avoids.

Geographic versus global cross-Region profile

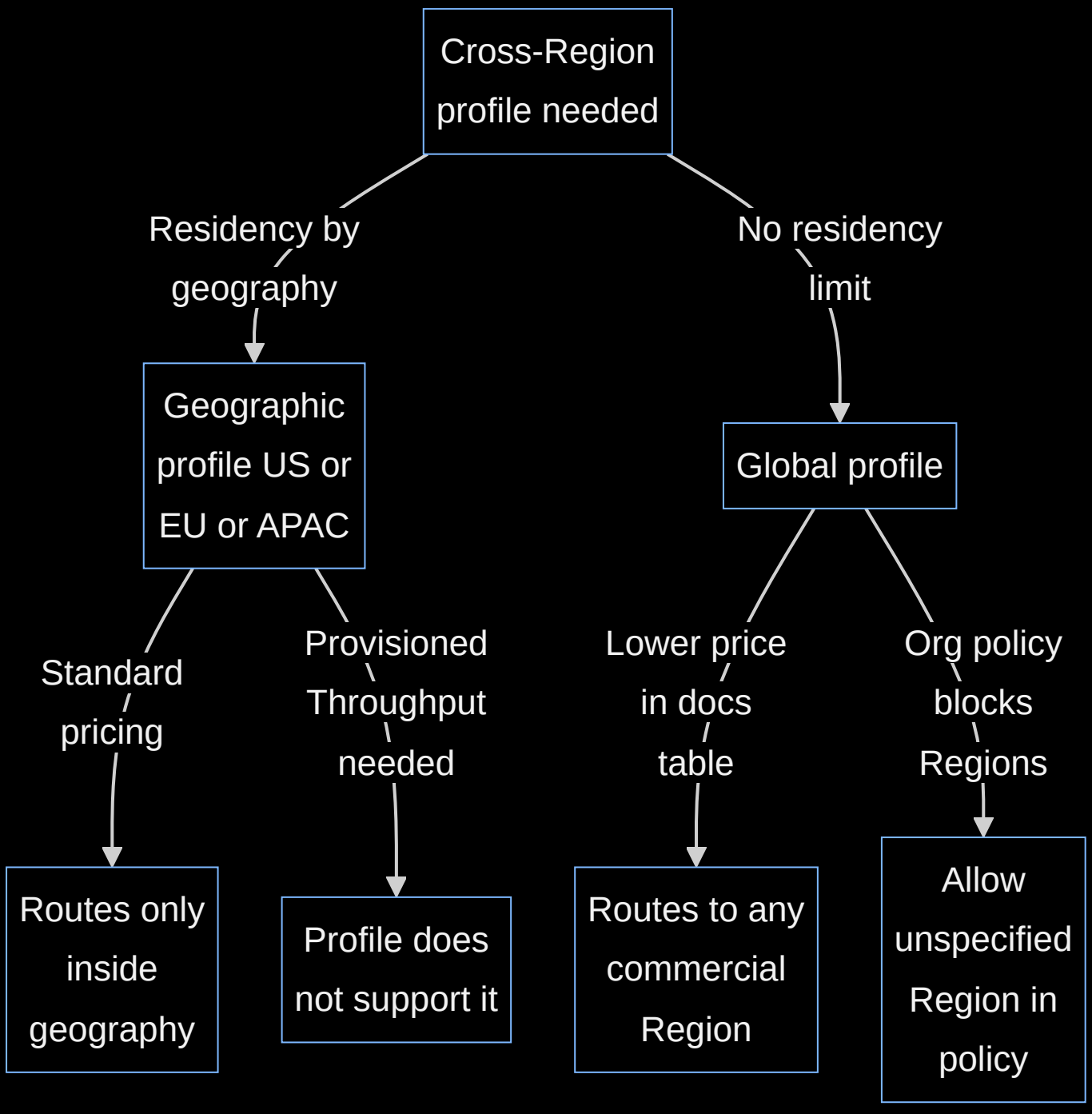


Figure: Residency language is the first filter, and price is the second.

- Purpose: A geographic profile keeps processing in a geography. A global profile maximizes the Regions available for routing.
- Input: Both take a profile ID or ARN as the modelId.
- Output or effect: Geographic has standard pricing and keeps data in the geography. Global is about 10 percent lower in the docs table and can route worldwide.

The one question: Does the rule name a geography or only a cost goal?

Trap: Choosing global because it is cheaper, when the stem names data residency. The cheaper option breaks the rule.

System-defined profile versus application inference profile

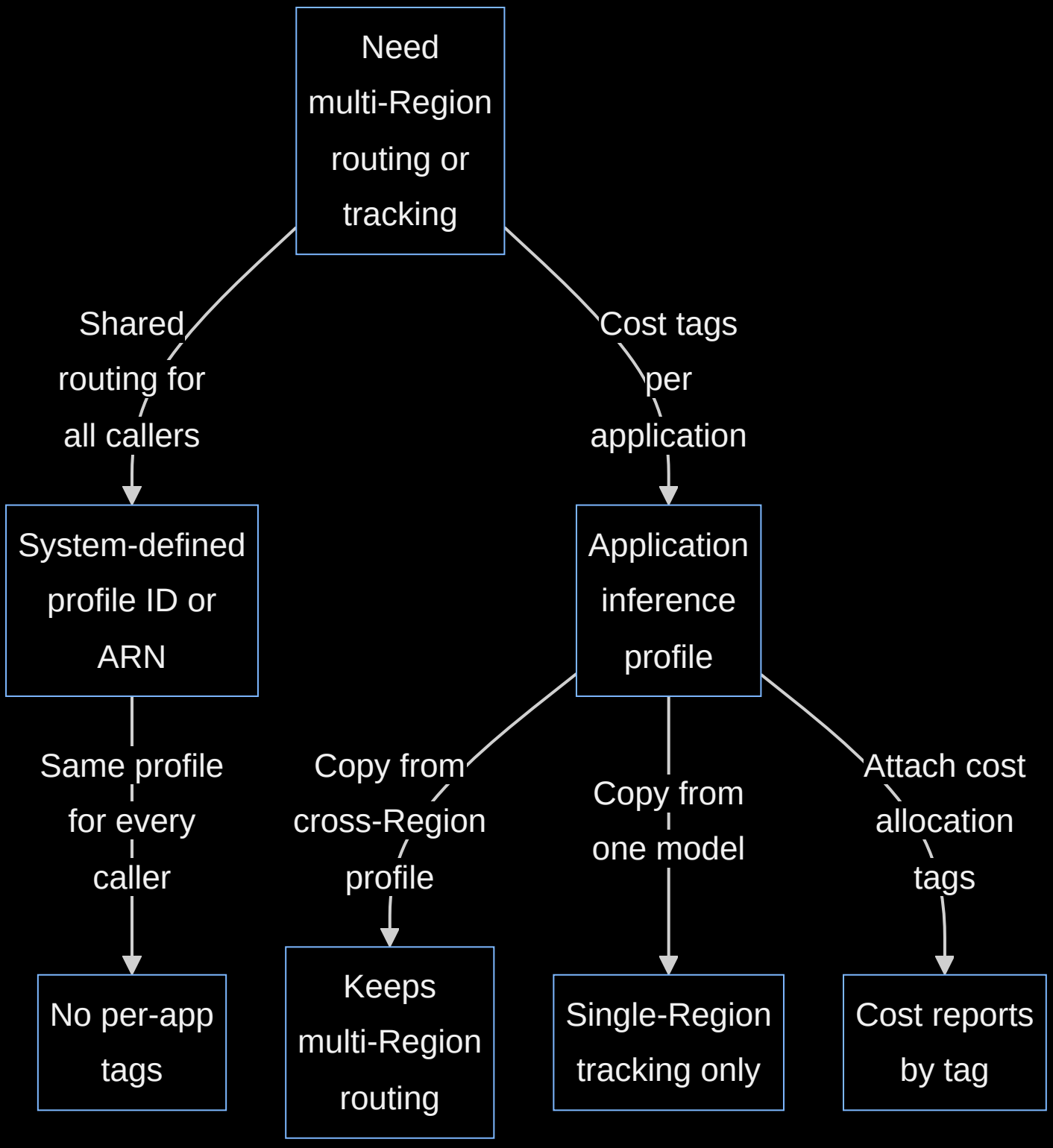


Figure: Tracking and routing are separate jobs, and a profile can do one or both.

- Purpose: The system-defined profile routes. The application profile routes and also labels usage for cost reports.
- Input: Each takes a profile ARN or ID as the modelId in the runtime call.
- Output or effect: The application profile returns its own ARN and tracks usage and cost for its Regions.

The one question: Do you need per-application cost, and does the copy keep multi-Region routing?

Trap: Copying an application profile from a single-Region model and expecting failover. The routing set stays at one Region.

Prompt caching versus semantic caching

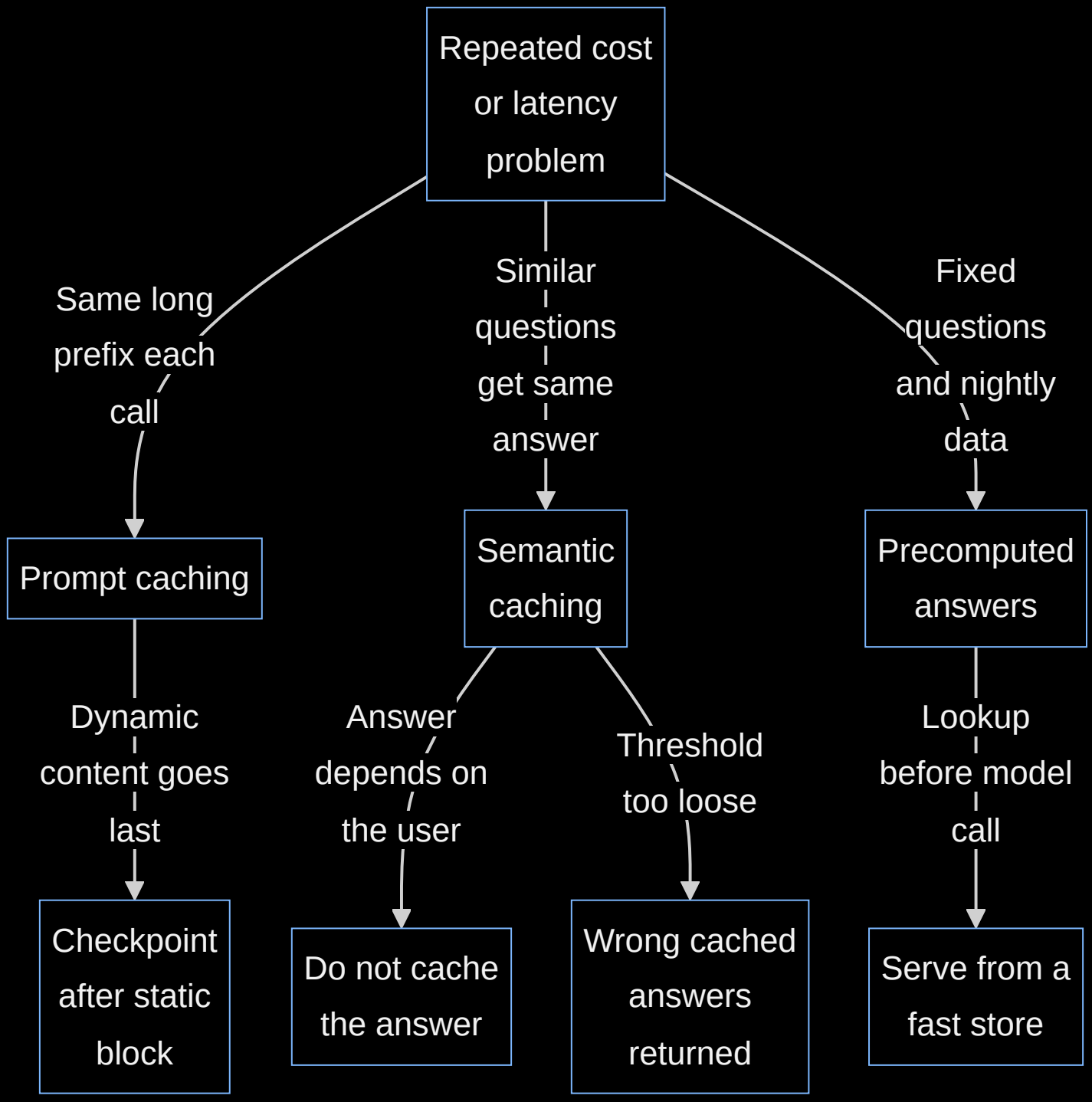


Figure: Prompt caching saves model work, and semantic caching skips the model.

- Purpose: Prompt caching reuses model input processing. Semantic caching reuses a final answer.

- Input: Prompt caching needs an identical prefix. Semantic caching needs an embedding of the new prompt.
- Output or effect: Prompt caching lowers input cost and latency when it hits. Semantic caching can skip the model call entirely when the similarity passes.

The one question: Is the expensive part the input that repeats, or the answer that repeats?

Trap: Expecting prompt caching to return the same answer for similar questions. It does not do that, and cache hits are not guaranteed.

Latency optimized inference versus streaming

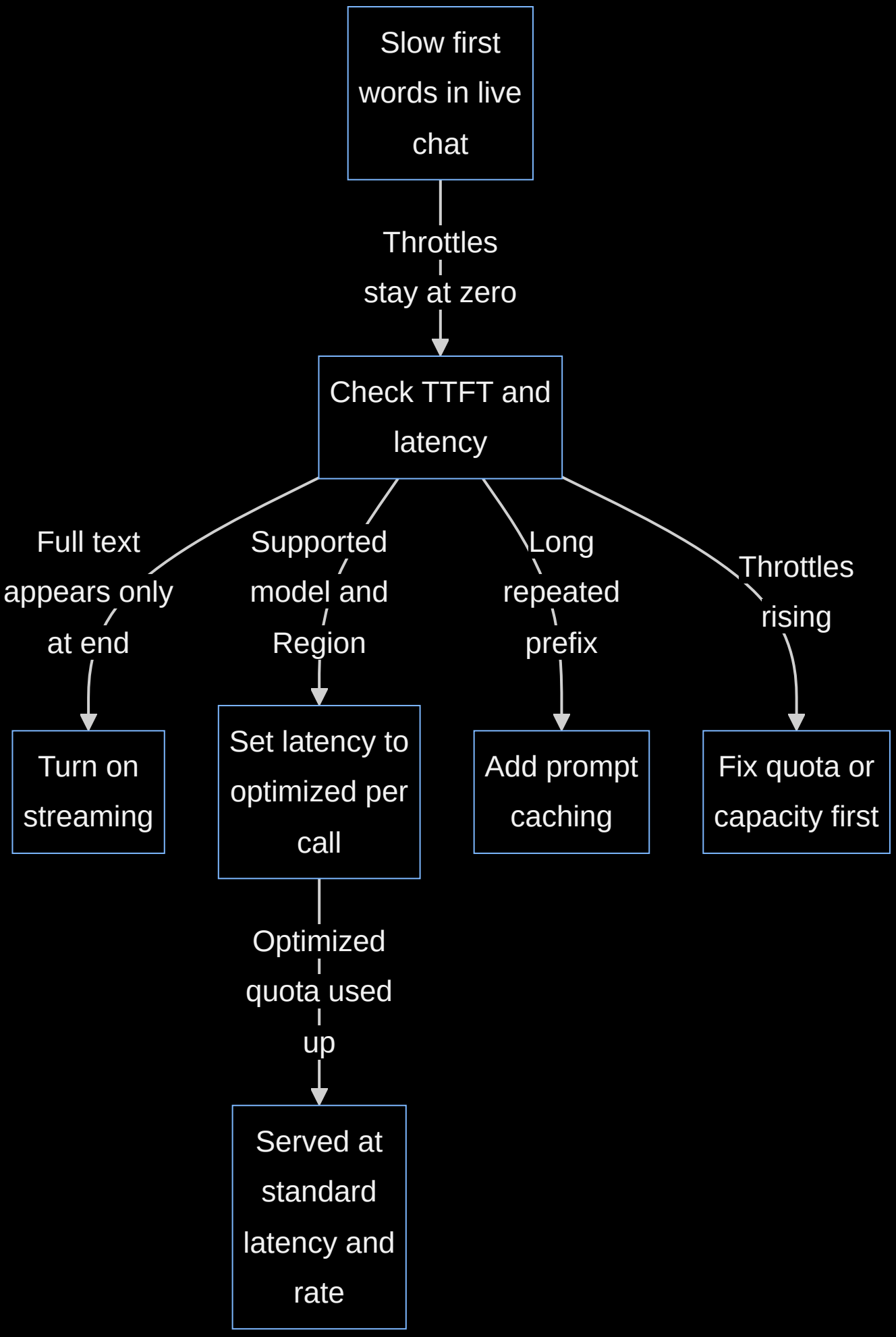


Figure: Streaming changes display timing, and optimized latency changes model serving speed.

- Purpose: Streaming shows output while it forms. Latency optimized inference asks the service for faster serving on a supported model.
- Input: Streaming uses ConverseStream or the streaming InvokeModel variant. Optimized latency uses performanceConfig on the call.
- Output or effect: Streaming lowers time to first visible text. Optimized latency can lower time to first token and total latency, but only where the feature applies.

The one question: Is the delay in the model or in when the user sees the text?

Trap: Buying Provisioned Throughput to fix first-token delay when there are no throttles. Capacity does not change how fast the first token arrives for a single call.

429 versus 503 and 529

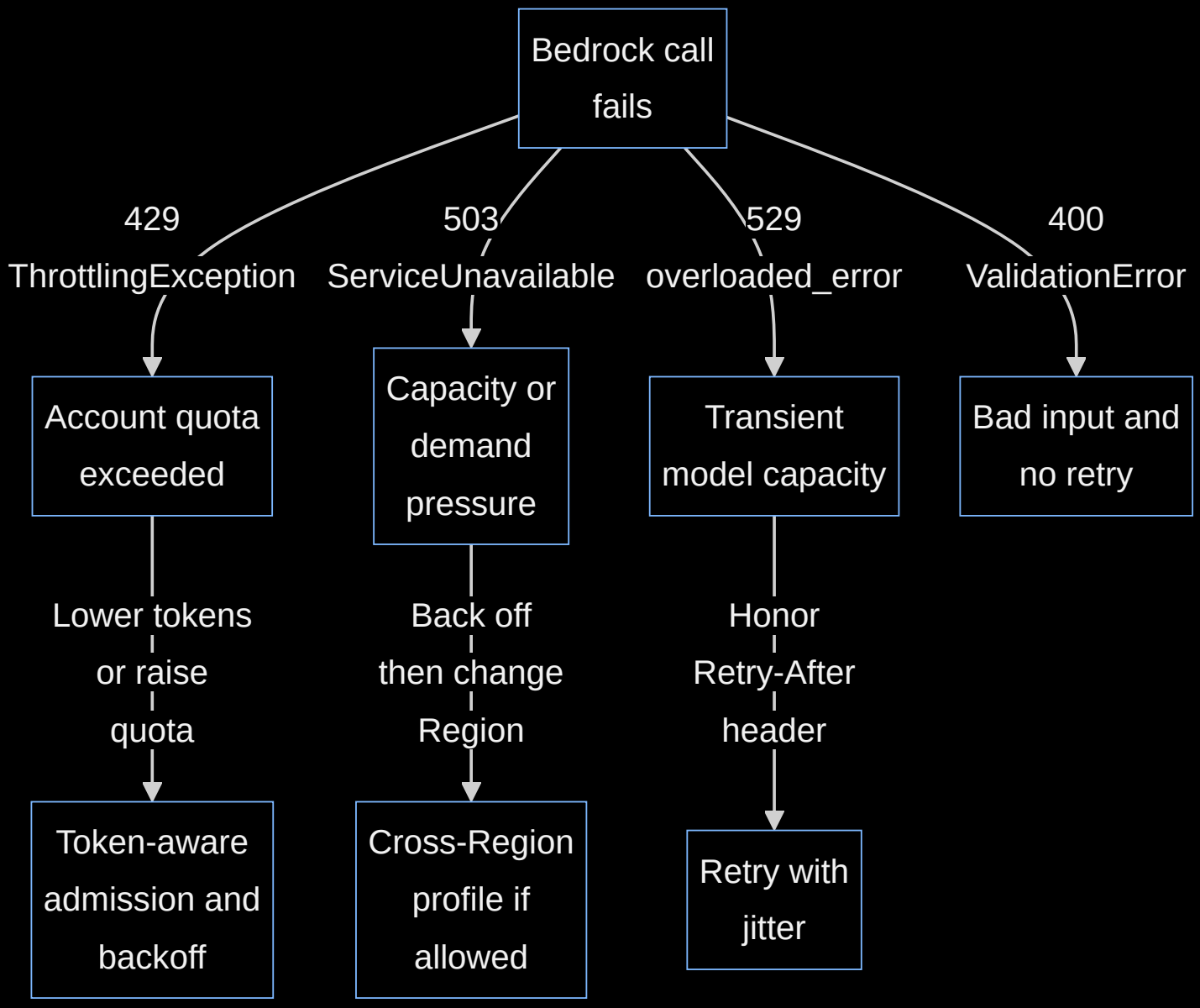


Figure: The status code separates your quota from the service capacity.

- Purpose: A 429 is your account limit. A 503 or 529 is service-side pressure.
- Input: The error type and the status code. Check the quota graph and the request token sizes.
- Output or effect: A 429 needs less token demand or more quota. A 503 or 529 needs backoff, lower concurrency, and sometimes another Region.

The one question: Is my account over its quota, or is the service busy for everyone?

Trap: Answering a 503 with a quota increase request. The quota increase does not change service-side capacity.

Read a long question

Each scenario below is written for practice. Underline the constraint words first. Then reject each wrong option by its type, and give one reason for each rejection. The answer comes last, so you can test yourself before you read it.

Scenario 1: Nightly backlog

A ticketing company summarizes forty thousand support threads each night. Nobody waits for a result, and the summaries must reach the warehouse by morning. The thread files already sit in S3, and the team does not want capacity that bills during the day.

- Constraint words: nobody waits, by morning, already in S3, no daytime capacity cost.
- Rejected type: a real-time loop with higher concurrency. Reason: it creates throttling and retry load for work that has no user.
- Rejected type: Provisioned Throughput. Reason: it bills every hour, which breaks the daytime cost rule.
- Rejected type: a self-hosted endpoint. Reason: it adds hosting work that the stem did not ask for.

Answer: Run a batch inference job with S3 input and S3 output.

Rule: A backlog in S3 with no waiting user points to batch.

Scenario 2: Residency with spikes

A public agency runs a citizen assistant and must keep all processing inside the European Union. Demand rises sharply near tax deadlines. The budget does not allow dedicated capacity.

- Constraint words: inside the European Union, sharp rises, no budget for dedicated capacity.
- Rejected type: a global profile. Reason: it can route outside the geography, which breaks the residency rule.

- Rejected type: one Region with a larger quota. Reason: a single Region still has one ceiling and cannot absorb the spike.
- Rejected type: Provisioned Throughput. Reason: the budget excludes it.

Answer: Use a European Union geographic profile as the modelId, with token-aware admission and bounded backoff.

Rule: A geography rule selects the geographic profile, not the global one.

Scenario 3: Slow first words, no throttling

A sales assistant answers in a chat window. Users complain that the first words take too long to appear. Throttling is zero, the prompts are short and unique, and the team must keep the same model.

- Constraint words: first words, zero throttling, short unique prompts, same model.
- Rejected type: Provisioned Throughput. Reason: the metrics show no throttling, so capacity is not the bottleneck.
- Rejected type: prompt caching. Reason: unique prompts give few cache hits, so the benefit is small.
- Rejected type: cross-Region routing with standard latency. Reason: it spreads load, but it does not target the first-token delay.

Answer: Stream the output and set latency to optimized on the live calls when the model and Region support it.

Rule: A first-token complaint with no throttling points to streaming and optimized latency.

Distractor patterns seen in the tests

These patterns repeat across questions in this area. Each one is a concept-level mistake, so learn the reason, not only the example.

- Trap: Right service, wrong capacity model. Provisioned capacity for a nightly job pays for idle hours. On-demand for a guaranteed latency target risks shared-capacity throttling.
- Trap: Retries as the fix for a quota. Backoff spreads the same demand over time. It does not add tokens per minute, and a quota-bound workload still fails.
- Trap: Concurrency as the fix for a token limit. A worker pool that counts requests ignores large prompts. Admission should count tokens.
- Trap: A larger output cap as a fix for missing input. maxTokens controls the reply length. It does not add context.
- Trap: CountTokens as an output predictor. The call counts input only, so output still needs a budget.
- Trap: A global profile for a residency rule. The cheaper option is wrong because the geography is the constraint.
- Trap: Hand-built failover when a managed profile exists. A custom router adds code and failure modes that the profile already covers.
- Trap: Prompt caching for answers. The feature reuses input processing, not final text.
- Trap: Semantic caching for personal answers. A similar question from another user can return the wrong account data.
- Trap: Metrics as a content audit. Metrics show counts and latency. Content review needs invocation logs.
- Trap: A cost tag on a request as a billing tag. Request metadata and application profile tags help attribution, and each has its own rules. Do not assume they are the same thing.
- Trap: Lambda concurrency as model capacity. The Lambda limit controls the callers. It does not raise Bedrock capacity.

Drill

Read the prompt, cover the answer line, and say the answer aloud before you read it.

Drill 1. A nightly job of 200,000 records in S3 must finish within 12 hours.

Rule: Run a batch inference job. Put JSONL records in S3 and read the results from S3.

Drill 2. A customized model must serve production traffic. Which capacity option is required?

Rule: Provisioned Throughput, and pass its provisioned model ARN as modelId.

Drill 3. The rule keeps processing inside the United States. The team wants the lowest price.

Trap: A global profile is cheaper but can route outside the United States. Use the US geographic profile.

Drill 4. Finance wants cost per team, and multi-Region routing must stay.

Rule: Create an application inference profile from the cross-Region profile, and attach the team tag.

Drill 5. Every call repeats the same policy text, and the user question comes last.

Rule: Use prompt caching with a checkpoint after the policy text.

Drill 6. Thousands of users ask near-identical questions, and the answer is the same for each of them.

Rule: Use semantic caching with a tuned similarity threshold and an expiry time.

Drill 7. Requests fail with 429 while the request count is under the limit, and prompts are large.

Trap: Raising the request-per-minute limit does not help. Check the token quota and the reserved input plus max_tokens.

Drill 8. A 503 spike happens at peak, and the quota graph is flat.

Rule: Back off with jitter, reduce concurrency, and use cross-Region routing when the rules allow it.

Drill 9. Answers stop mid-sentence, and the stop reason is max_tokens.

Rule: Raise or tune the output cap for that task class. Input overflow is a separate case.

Drill 10. You need the input size of a full request before you call the model.

Rule: Call CountTokens on the same model with the full assembled request.

Drill 11. Which metric shows the first-token delay of a chat stream?

Rule: TimeToFirstToken, because it applies to streaming calls.

Drill 12. Which record shows which IAM identity called the API and when?

Trap: CloudWatch metrics do not identify callers. Use CloudTrail for the identity and the time of the call.

Sources

- https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/batch-inference.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/cross-region-inference.html

- https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-routing.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/quotas.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/quotas-token-burndown.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/latency-optimized-inference.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/troubleshooting-api-error-codes.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/scaling-throughput-best-practices.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/inference-profiles-create.html
- Sundog course slides, pages 21, 225, 285, 286, 288 to 295, 304, 305, 319, 446, 571, 608, and 609 (second source of facts for this module)